

KREACIJSKI DIZAJN PATERNI

Grupa 6 - Tim2

1 Singleton

Uloga Singleton paterna je da osigura da se klasa može instancirati samo jednom i da osigura globalni pristup kreiranoj instanci. Primjeri korištenja singleton paterna u sistemima su za praćenje logova, objekti za dražvere raznih uređaja poput printera, grafičke kartice itd..

Kreiranjem singleton klase, definiranjem privatne statičke read-only instance te singleton klase, postavljanjem privatne vidljivosti konstruktora, dodavanjem public metode koja pristupa privatnom objektu i instancira ga (**samo ako ne postoji**), možemo implementirati sistem centralizacije toka slanja mailova u našem sistemu,.

U našem sistemu imamo različite tipove notifikacija: notifikacije za verifikaciju firme, i notifikacije koje su vezane za oglase na našem sistemu. Kako se ne bi svaki put stvarala nova instanca klase za slanje (EmailVerifikacijaFirme i EmailNotifikacijaOglas, koje preko IEmailSender implementiraju metodu PosaljiEmail()) koristimo Singleton obrazac koji vrši centralizaciju svih tipova notifikacija u jednom queue-u instance singleton klase EmailNotificationQueue.

Ovim pristupom stičemo prednosti jednostavnog upravljanja svim mailovima u sistemu zbog centralizacije, manjak redundancije u smislu postojanja samo jedne instance klase EmailNotificationQueue tokom cijelog životnog vijeka aplikacije, a ne više instanci klasa za različite tipove mailova, spriječavanje pojave race conditiona, jednostavno proširenje sistema dodavanjem novih funkcionalnosti (npr. dodavanje klase za slanje mailova za reset lozinke) i mnoge druge.

2 Factory Method

Factory Method patern omogućava kreiranje objekata na način da podklase, na osnovu informacije od strane klijenta ili tekućeg stanja sistema, odluče koju klasu instancirati. Ovaj patern odvaja zahtijevanje objekata od njihovog kreiranja klijent radi isključivo sa apstrakcijom i ne mora poznavati konkretan tip kreiranog objekta.

U našem sistemu ovaj patern ima direktnu primjenu pri kreiranju oglasa. Sistem sadrži tri tipa oglasa sa različitim atributima i kontekstom kreiranja: OglasRadnoMjesto kreira Firma, OglasUsluge kreira Klijent, a OglasMajstora kreira Majstor.

Bez Factory Method paterna, logika odlučivanja koji oglas kreirati bila bi rasuta po cijeloj aplikaciji u if blokovima, što narušava OCP, svaki novi tip oglasa zahtijevao bi izmjenu svih tih mjesta.

To možemo implementirati:

- definisanjem interfejsa IOglasKreator koji sadrži Factory Method KreirajOglas(): Oglas;

- definisanjem konkretne klase RadnoMjestoKreator koja implementira IOglasKreator i instancira OglasRadnoMjesto;
- definisanjem konkretne klase UslugeKreator koja implementira IOglasKreator i instancira OglasUsluge;
- definisanjem konkretne klase MajstorOglasKreator koja implementira IOglasKreator i instancira OglasMajstora;
- svaki konkretan kreator veže se za odgovarajući tip korisnika — Firma koristi RadnoMjestoKreator, Klijent koristi UslugeKreator, Majstor koristi MajstorOglasKreator.

Korištenje:

Prilikom kreiranja oglasa, klijentski kod poziva metodu KreirajOglas() kroz interfejs IOglasKreator, ne znajući koji konkretan tip oglasa će biti instanciran.

Logika odlučivanja smještena je unutar konkretnih kreatora, a ne u kontrolerima i servisima. Ukoliko bi u budućnosti bio potreban novi tip oglasa, dovoljan je novi kreator bez izmjene postojećeg koda, čime se u potpunosti poštuje OCP.

3 Prototype

Prototype patern obezbjeđuje interfejs za konstrukciju objekata kloniranjem jedne od postojećih prototip instanci i modifikacijom te kopije. Koristi se kada je kreiranje novog objekta resursno zahtjevno ili kada novi objekti u većini slučajeva dijele iste vrijednosti atributa.

Razmatrajući naš sistem, svaki oglas, recenzija ili ponuda usluge predstavljaju jedinstvene korisničke podatke svaki ima drugog vlasnika, drugi datum, drugačiji sadržaj. Kloniranje postojećeg oglasa radi kreiranja novog nema smisla jer bi se morali promijeniti gotovo svi atributi. Stoga ovaj patern nije opravdan.

Korištenje:

Prototype patern se ne primjenjuje u ovom sistemu jer ne postoje objekti čije bi kloniranje donijelo značajnu uštedu resursa ili vremena. Svi entiteti sistema su jedinstveni i zahtijevaju individualno kreiranje.

4 Abstract Factory

Abstract Factory patern omogućava kreiranje niza međusobno povezanih objekata bez specificiranja konkretnih klasa. Koristi se kada postoji hijerarhija objekata koja objedinjuje različite platforme sastavljene od grupe povezanih objekata.

Razmatrajući naš sistem, Abstract Factory bi bio opravdan kada bi postojale različite varijacije korisnika ili oglasa za različite platforme. Međutim, sistem je fokusiran na jednu platformu nema niza međusobno zavisnih objekata čije bi kreiranje trebalo enkapsulirati.

Korištenje:

Abstract Factory patern se ne primjenjuje u ovom sistemu jer ne postoje grupe međusobno zavisnih produkata niti višestruke platforme koje bi zahtijevale odvojene kreatore. Uvođenje ovog paternu donijelo bi nepotrebnu kompleksnost bez razloga.

5 Builder

Builder dizajn patern koristan je za kreiranje složenih objekata korak po korak. Primjenjuje se kada objekat ima mnogo parametara ili kada je potrebna fleksibilnost u kreiranju objekta, a cilj je odvojiti specifikaciju objekta od njegove stvarne konstrukcije.

Posmatrajući naš sistem, OglasRadnoMjesto sadrži veći broj atributa — uslovi zaposlenja, vozačke dozvole, tip isplate, minimalni i maksimalni prihod. Međutim, ovi atributi su uglavnom obavezni i postavljaju se odjednom pri kreiranju oglasa, a ne postepeno korak po korak. Factory Method efikasnije rješava problem kreiranja ovih oglasa.

Korištenje:

Builder patern se ne primjenjuje u ovom sistemu jer ne postoje objekti koji zahtijevaju postepenu izgradnju kroz više odvojenih koraka. Kreiranje oglasa i ostalih dijelova odvija se odjednom, što čini Factory Method mogućim rješenjem.